

# Property D: The LLM Does Not Exercise Authority Over Substrate Content — Standalone Architectural Commitment in the Coordination Knowledge Substrate Pattern

**Author:** Wenxin Li (Independent Researcher)

**ORCID:** 0009-0004-8065-3235

**Date:** 2 May 2026

**License:** Creative Commons Attribution 4.0 International (CC BY 4.0)

---

## Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one of the five mediator properties named in the source paper’s AI-as-substrate-mediator commitment — the property that **the LLM does not exercise authority over substrate content** (Property D) — as a standalone architectural commitment with independent operational content, separable from the four sibling properties with which it composes.

## Abstract

The Coordination Knowledge Substrate (CKS) pattern’s AI-as-substrate-mediator commitment names five properties that together specify the architectural content of the LLM’s role in a CKS-coherent system. A separate parent-frame note (A2.18) formalizes the five-property set as severable architectural commitments. This note formalizes one of the five — Property D, the commitment that the LLM does not exercise authority over substrate content — as a standalone commitment with independent operational content. Property D is the negative specification of LLM write behavior, paired with Property B (the positive specification: LLM writes happen under orchestration rules); together they fix the architectural content of LLM operations against the substrate as rule-governed labor under human authority, not autonomous authority claims. The note identifies five operational components Property D requires, distinguishes it from four adjacent patterns commonly conflated with it, names eight failure modes that violate it specifically, traces five load-bearing connections to other CKS commitments, and provides an operational test for whether a system’s LLM authority behavior is CKS-coherent on this axis.

## 1. Why Property D needs to be formalized as standalone

The CKS pattern’s AI-as-substrate-mediator commitment (§4.1, §4.2 of the source paper) specifies the LLM’s architectural role through five properties: the LLM reads from substrate as primary source of state (Property A), writes under orchestration rules (Property B), does not hold substrate-relevant state outside substrate (Property C), does not exercise authority over substrate content (Property D), and produces substrate-affecting outputs that are recorded in substrate with attribution (Property E). The parent-frame note A2.18 establishes the severable-set structure these five properties form. Each property carries operational content of its own and admits a standalone derivation note; this is the fourth such treatment, after A2.19 (Property A), A2.20 (Property B), and A2.21 (Property C).

The need for the standalone treatment is concrete. Implementations under pressure to support sophisticated LLM behavior consistently drift toward LLMs exercising authority through emergent patterns: agent-framework deployments where the LLM acts on its own judgment about substrate modifications; LLM “improvement” tools that deduplicate, summarize, or reorganize substrate content without explicit rule authorization; LLM mediators that resolve detected contradictions by selecting a winner outside the cell-level resolution mechanism the rules specify. Each is a specific authority claim — and each is invisible if the integrated mediator role is treated as a single composite without the five properties separated.

A second motivation is the strategic prior-art posture for downstream patentable derivations. Derivations in the LLM-authority space — autonomous-agent designs, LLM-driven governance features, LLM-judgment conflict handling — are substantially more defensibly contested when Property D is publicly formalized as standalone, because any “LLM authority innovation” can be evaluated against the specific architectural commitment to no-LLM-authority-over-substrate.

The third and most architectural motivation is the connection to the authority-vs-labor distinction A1.01 formalizes. That distinction is what makes the human-governed commitment defensible against the central misreading the source paper preempts (§3.3): governance is an authority architecture, not a review workflow. Property D operationalizes the distinction at the LLM layer. The LLM exercises labor extensively — reading, processing, writing, mediating — but does not exercise authority. Without Property D as standalone, the authority-vs-labor distinction collapses at the LLM boundary, and LLM behavior drifts toward exercising the authority humans hold, taking substrate governance with it.

## 2. The Property D commitment, defined precisely

In the CKS pattern, a system satisfies Property D if and only if all of the following hold during LLM execution within cells.

**(a) The LLM does not silently overwrite or delete substrate content.** Substrate writes from LLM operations happen under orchestration rules (Property B, A2.20); LLM operations that modify substrate content outside rule authorization, even when the modification is well-intended (deduplication, normalization, “improvement”), violate Property D. The architectural content is not the modification but the absence of rule authorization for it.

**(b) The LLM does not silently collapse contradictions in substrate content.** When the LLM encounters contradicting substrate content during execution, the resolution happens under cell-level orchestration rules (per A2.14), with the resolution decision recorded as substrate content (per A2.15). LLM operations that proceed with one interpretation of contradicting content while ignoring the other, without an orchestration rule specifying the response, violate Property D. The source paper states this directly (§5.3): the architectural constraint is not on whether conflicts get resolved, but on who decides resolution logic; humans decide, the LLM does not.

**(c) The LLM does not modify orchestration rules.** Rule authorship is a human exercise (per A2.04). LLM operations that propose rule changes are admissible — the proposal is then evaluated by humans who hold authority. LLM operations that commit rule changes to substrate without human authority being exercised in the commit violate Property D.

**(d) The LLM does not exercise the override right.** The override right belongs to humans with override authority (per A2.03). LLM operations that take override actions — changing substrate content outside rule

constraints, modifying rules mid-execution, undoing or replacing cell decisions outside rule authorization — violate Property D, even when the override would have been justified if a human had exercised it. The justification is not the right; the authority to act on it without justification is.

**(e) The LLM does not exercise authority over the architecture itself.** Decisions about which cells exist, which rules apply to which cells, how the substrate is structured, and how the host environment is configured are architectural and deployment decisions. The source paper locates these with humans through Claim 2’s schema-level decomposition (§4.1) and Claim 5’s tool-agnosticism (§7); they are deployment-time human authority. LLM operations that take such decisions outside human authorization extend the Property D violation to the architecture itself. This component is a derivation from the source paper’s Claim 2 and Claim 5 commitments rather than a freestanding axiom; it makes explicit at the LLM layer what the source paper locates with humans at the deployment layer.

The five components together define what Property D requires. A system that satisfies fewer than five permits the LLM to exercise authority humans hold in some way, breaking the architectural commitment.

### 3. What Property D does NOT require

The standalone treatment is not a maximalist treatment. Stating precisely what the property does not require is what keeps the commitment from drifting into a constraint on LLM sophistication or labor.

**It does not require the LLM to be passive or unsophisticated.** The LLM may produce nuanced reasoning, complex multi-step analysis, contextually-aware responses, and creative solutions. None of this violates Property D as long as the outputs do not claim authority. Sophistication is a property of the LLM’s labor, not its authority.

**It does not require the LLM to defer to humans on every decision.** Within rule authorization (Property B, A2.20), the LLM produces outputs without per-execution human approval. Property D forecloses authority claims; it does not impose a per-operation review checkpoint. The labor allocation framework (A1.12) supports modes in which the LLM does substantial work autonomously, and Property D is compatible with all such modes provided the work does not claim authority.

**It does not forbid the LLM from suggesting changes.** The LLM may propose substrate modifications, recommend rule changes, flag contradictions for human review, or surface considerations that suggest specific actions. Suggestions are inputs to humans who hold authority. The architectural commitment is to humans deciding under their authority, not to humans receiving no LLM input.

**It does not require the LLM to be deterministic.** LLMs are non-deterministic by design (A1.10’s representation-determinism vs. model-output-determinism distinction); Property D does not change this. What Property D constrains is the LLM’s authority, not its predictability. An LLM producing different outputs across executions may still satisfy Property D as long as each output is rule-governed (Property B) and does not claim authority (Property D).

**It does not forbid the LLM from operating in domains where humans have delegated substantial labor to LLMs.** The labor allocation framework (A1.12) supports Mode 2 (LLM under rule) and Mode 3 (stable-cell automation) where the LLM does substantial work. What the framework does not support is the LLM also doing authority work — labor is delegated; authority is not.

## 4. What Property D is NOT

Four adjacent patterns are commonly conflated with Property D. Each is a real and reasonable design move; naming what Property D is not is what prevents the misreading.

**Not a prohibition on LLM-mediated conflict resolution under rules.** The cell-level conflict resolution mechanism (A2.14) uses the LLM to apply orchestration rules at execution time when contradictions arise. The LLM is doing the resolution work; the authority is in the rule. This is fully Property D-compliant. What violates Property D is silent resolution outside rules, not rule-mediated resolution by the LLM.

**Not a prohibition on LLM-decided write content.** Within rule authorization (Property B), the LLM decides the specific content of its writes — the words, structure, and reasoning the write contains. This is execution within rule authorization, not authority over substrate content in the architectural sense. Property D forbids authority that humans hold; it does not forbid the LLM from making content choices its rule authorizes.

**Not a prohibition on autonomous reasoning during execution.** During a single cell execution, the LLM may reason autonomously — exploring multiple paths, considering alternatives, drawing inferential connections. Autonomous reasoning is execution-internal labor; it does not constitute authority over substrate content. The distinction matters in 2024–2026 LLM infrastructure, where autonomous-agent frameworks (agent loops with tool calls, planning agents, multi-step reasoning chains) are the dominant pattern. The LLM operating inside such a framework may exercise autonomous reasoning across many steps without violating Property D, provided each substrate-affecting step is rule-authorized (B) and none of them claim authority humans hold (D). Autonomous *reasoning* is execution-internal labor; autonomous *agency* — cross-execution authority claims, in which the LLM’s outputs are treated as governance moves rather than rule-governed operations — is the Property D violation. The two collapse easily under casual use of the term “autonomous”; the standalone treatment of D is what keeps them distinct.

**Not a prohibition on LLM-suggested rule changes.** The LLM may analyze patterns in substrate content and orchestration rules, propose rule modifications, surface gaps in rule coverage, or recommend rule deprecation. These are suggestions humans evaluate. The architectural commitment is that humans commit rule changes (A2.04); LLMs may propose, but the commit is human authority. Suggestions do not violate Property D; commits without human authority do.

## 5. Why Property D is load-bearing for downstream commitments

Property D is load-bearing for several other CKS commitments, and naming the connections briefly clarifies why the standalone treatment matters.

*The human-governed commitment (A1.01).* Human governance at the LLM layer is operationalized through Property D. Without Property D, the LLM exercises authority that belongs to humans; with Property D, the LLM is bounded to labor while humans retain authority. Property D is what makes the authority-vs-labor distinction architecturally specific at the LLM boundary.

*The conflict preservation commitment (A1.03).* The commitment depends on contradictions not being silently collapsed by automated processes. Property D forecloses LLM-silent-collapse specifically (component (b) of §2 above); cell-level resolution under rules (A2.14) provides the architectural mechanism by which contradictions are handled at execution time without LLM authority being exercised.

*The architectural-property qualifier on governance (A2.06).* The architectural-vs-procedural distinction requires that governance is a property of architecture, not of process. Property D contributes by making LLM-authority-limitation architectural: the LLM cannot exercise authority as a property of the architecture's design, regardless of how the LLM is deployed or what process layers exist around it.

*The orchestration-layer distinction (A1.15).* The distinction between coordination-knowledge layer (CKS) and coordination-mechanism layer (workflow engines, agent frameworks) depends on LLMs not exercising authority over substrate content. Without Property D, the layers blur: the LLM operating within a cell could exercise authority as if it were an autonomous agent, collapsing the architectural distinction the layer model preserves.

*The override right's standalone status (A2.03).* The override right belongs to humans. Property D forecloses LLMs from exercising it, which is what makes the override right's human-only status architectural rather than conventional.

## 6. Failure modes that violate Property D

Eight anti-patterns name ways an implementation can fail this commitment specifically. Each is a distinct way an LLM crosses from labor into authority.

**(a) Autonomous LLM modifications.** The LLM produces substrate modifications based on its own judgment, outside orchestration rule authorization. The modifications may be well-intended (deduplication, normalization, improvement), but they exercise authority humans hold over substrate content.

**(b) Silent conflict resolution by LLM judgment.** The LLM encounters contradicting substrate content and proceeds with one interpretation, ignoring the other, without an orchestration rule specifying the response. The LLM has exercised the authority that the cell-level resolution mechanism reserves to rules.

**(c) LLM-driven rule changes.** The LLM commits modifications to orchestration rules without human authority being exercised in the commit. The rules may be improved by the changes, but the commit is the LLM's act, not a human's act, which violates the rule-authoring commitment from A2.04.

**(d) LLM-exercised override.** The LLM takes actions that override orchestration rules, undo cell decisions, or modify substrate content outside rule constraints, treating itself as having override authority. The override right belongs to humans (A2.03); the LLM's exercise of it is a Property D violation regardless of whether the override would have been justified if exercised by a human.

**(e) Implicit authority through prompt design.** The system uses prompts that grant the LLM authority within the prompt's framing — “you may modify substrate content as you see fit,” “decide how to handle this contradiction,” “improve the coordination state.” The prompt is not a substrate-resident orchestration rule; the authority granted by the prompt is not architectural. The LLM operating under such prompts exercises authority outside the architecture.

**(f) LLM-as-arbiter pattern.** The system positions the LLM as the decision-maker among humans with conflicting authority — the LLM “decides” who wins when humans disagree, or “judges” which human's substrate write should prevail. The LLM has been given authority over human authority, which violates Property D in a particularly subtle way: the LLM is governing the governors.

**(g) LLM-driven workflow control.** The LLM decides which cells execute, in what order, with what scope. These are architectural decisions belonging to humans (cell design, orchestration rule design, workflow design per Claim 2). LLM operations that take such decisions outside human authorization violate Property D’s architectural-decisions component.

**(h) LLM “self-improvement” patterns.** The LLM is granted authority to modify its own orchestration rules, expand its own scope, or change its own role based on observed performance. The LLM has been given authority over its own architectural position, which is the strongest form of Property D violation — the LLM is now exercising authority over the architecture itself.

## 7. Operational test

A system satisfies Property D if and only if all of the following are true at all times during LLM execution within cells:

1. No LLM operation modifies substrate content outside orchestration rule authorization.
2. No LLM operation silently collapses contradictions in substrate content; conflict handling happens under rule-governed cell-level resolution per A2.14.
3. No LLM operation commits modifications to orchestration rules; rule changes happen through human authorship per A2.04.
4. No LLM operation takes override actions; the override right belongs to humans per A2.03.
5. No LLM operation takes architectural decisions (cell design, scope changes, role modifications) outside human authorization.

A system that fails any of (1)–(5) does not satisfy Property D in the architectural sense, even if its other mediator properties (A, B, C, E) are preserved.

## 8. Why naming Property D as standalone matters

Implementations under pressure to support sophisticated LLM behavior, autonomous-agent patterns, or “smart” coordination systems consistently drift toward LLMs exercising authority. The drift is steady because authority feels like sophistication: the LLM that “decides” appears more capable than the LLM that proposes; the LLM that “resolves” appears more useful than the LLM that surfaces. Each apparent capability gain is actually a transfer of authority from humans to LLM, breaking the architectural commitment to human governance over substrate content.

Implementations that drift away from Property D produce systems where the LLM is operationally indistinguishable from an autonomous agent, with the substrate playing a ceremonial role while the LLM exercises governance. The downstream consequences manifest as governance failures (humans cannot govern decisions the LLM made on its own authority), conflict-handling failures (the LLM silently resolves what should have been preserved), and architectural drift (the system gradually becomes an autonomous-agent system with CKS vocabulary rather than a CKS system).

Naming Property D as a standalone architectural commitment gives downstream implementers a precise specification of what Property D requires architecturally and what it does not. The remaining note in the AI-as-substrate-mediator decomposition (A2.23) formalizes Property E; with A2.23 complete, the decomposition will be fully formalized as five severable commitments under the parent foundational note

A1.04.

---

### **Source paper**

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

### **How to cite this note**

Li, W. (2026). *Property D: The LLM Does Not Exercise Authority Over Substrate Content — Standalone Architectural Commitment in the Coordination Knowledge Substrate Pattern*. 2 May 2026. ORCID: 0009-0004-8065-3235.